

Research paper

Deep reinforcement learning for periodic Unmanned Aerial Vehicle task allocation and path planning with a Fixed Nest Station in power inspection

Yuxuan Sun ^{a,b}, Feng Gao ^{a,b}, Yongjie Xue ^c, Bin Yu ^{a,b} ^{*}

^a Key Laboratory of Intelligent Transportation Technology and System, Ministry of Education, School of Transportation Science and Engineering, Beihang University, Beijing 100191, China

^b School of Transportation Science and Engineering, Beihang University, Beijing 100191, China

^c Department of Civil Engineering, University of Hong Kong, Hong Kong, China

ARTICLE INFO

Keywords:

Deep reinforcement learning
Unmanned Aerial Vehicle
Periodic power inspection
Task allocation
Path planning

ABSTRACT

In power system inspections, the coupling of heterogeneous inspection frequencies and strict endurance constraints necessitates complex long-horizon periodic scheduling. While traditional optimization methods often overlook these specific periodic requirements, the Unmanned Aerial Vehicle (UAV) Fixed Nest Station mode offers a new operational paradigm to execute such autonomous tasks. We consider this operational scenario and propose a specialized dynamic-pruning and topology-aware deep reinforcement learning (DPTARL) framework. The DPTARL is developed to mitigate myopic optimization behavior and improve the robustness of long-horizon periodic scheduling. Specifically, we construct a dynamic state pruning (DSP) architecture conditioned on system states. This architecture enforces feasibility constraints during decision making. As a result, it removes invalid actions and accelerates convergence. Furthermore, we incorporate a topology-aware attention (TAA) module to capture sequential dependencies among heterogeneous targets, enabling efficient allocation of periodic tasks. To address the challenges of long-horizon planning during training, we design a composite reward mechanism. It balances immediate inspection gains with long-horizon periodic consistency. This design helps reduce decision tendencies of myopic optimization behavior. Extensive computational experiments demonstrate the effectiveness of DPTARL. In small-scale verification, it achieves near-optimal performance with an optimality gap below 10% and substantial inference acceleration over exact solvers. In large-scale evaluations against representative baselines, including ALNS, PPO, and DQN, DPTARL reduces myopic optimization tendencies, shortens the average flight path by 16.64%, and improves flight efficiency by 9.89%. These results indicate that DPTARL provides an effective and scalable solution for large-scale, periodic and long-horizon UAV inspection scheduling.

1. Introduction

Ensuring the reliability of power systems requires regular and effective inspection. In real-world power inspection operations, inspection targets mainly consist of substations, transmission towers, and distribution poles. Due to distinct operational characteristics and exposure to environmental stress, power infrastructure components require different inspection frequencies (Mbuli et al., 2020; Cacace et al., 2021). For example, substations are typically inspected daily, transmission towers weekly, and distribution poles monthly. However, existing research often simplifies the reality of inspection by assuming uniform inspection requirements or by focusing on single trip path planning (Gazzea et al., 2022). Such assumptions ignore the periodic nature of inspection tasks and limit the applicability of the resulting methods to long-horizon maintenance in large-scale power networks.

Compared with manual inspections, unmanned aerial vehicle (UAV) based inspection has been widely recognized as a safer and more cost-effective solution for power infrastructure inspection (Liu et al., 2019; Tedeschi et al., 2020; Caballero et al., 2024). To enable autonomous and continuous inspection operations, the UAV Fixed Nest Station mode has recently emerged as a promising operational paradigm. By providing automated charging and storage capabilities, such infrastructure supports long-horizon planning of periodic missions with minimal human intervention. However, this operating mode poses a critical challenge due to tight coupling between UAV endurance limits and multi-frequency inspection demands. Unlike single trip optimization settings, decision making in this context exhibits strong temporal dependencies. As a result, decisions made at one stage of the schedule

* Corresponding author.

E-mail address: yubinyb@buaa.edu.cn (B. Yu).

<https://doi.org/10.1016/j.engappai.2026.115219>

Received 8 January 2026; Received in revised form 26 April 2026; Accepted 21 May 2026

Available online 28 May 2026

0952-1976/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

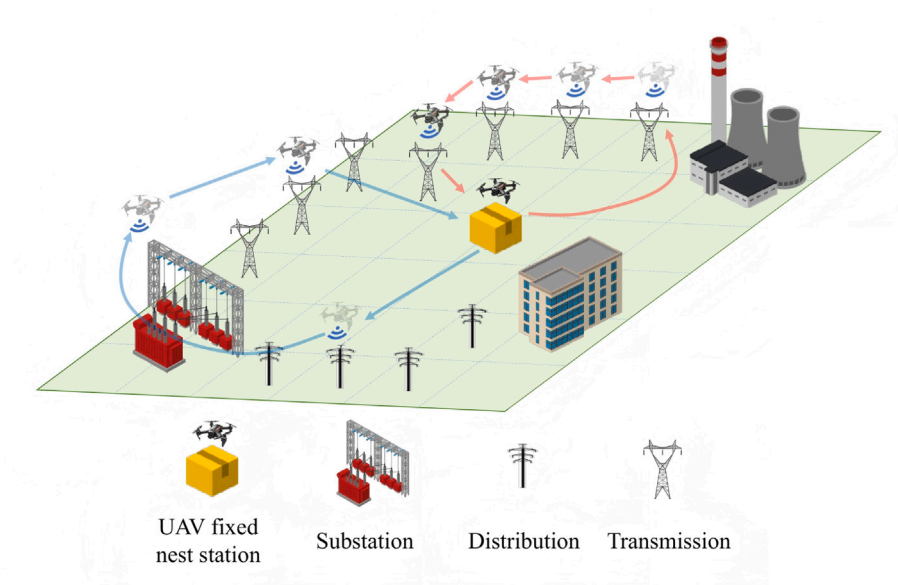


Fig. 1. Illustration of UAV Fixed Nest Station power inspection scenario.

can directly affect task feasibility and inspection compliance at later stages.

Solving the Periodic Power Inspection Problem under the UAV Fixed Nest Station mode (PIPUFN) entails three fundamental mathematical and computational challenges. First, the combination of long-horizon planning and large-scale power networks results in a combinatorial explosion of the state-action space, which makes traditional optimization methods difficult to scale. Second, inspection targets are spatially distributed and topologically heterogeneous, making it difficult for conventional solvers to capture the complex sequential dependencies among diverse nodes. Third, under strict endurance constraints, the scheduler may favor immediately reachable tasks and neglect future periodic requirements. In this paper, we describe this tendency as myopic optimization behavior. Such myopic optimization behavior ultimately leads to degradation of system stability and efficiency in the scheduling horizon. To evaluate whether such behavior is alleviated, we use flight efficiency (FE) as a practical indicator which detailed in Section 5. In this engineering environment, higher flight efficiency represents better task execution efficiency and indicates that myopic optimization behavior is suppressed. The application scenario is shown in Fig. 1.

To address the above challenges, we propose a dynamic-pruning and topology-aware deep reinforcement learning (DPTARL) framework for the joint optimization of periodic task allocation and path planning. The DPTARL incorporates operational feasibility constraints into the learning process and aims to produce effective scheduling policies over long planning horizons. It also balances immediate decision feasibility with long-term periodic requirements. The main contributions of this paper are summarized as follows:

(1) We propose a dynamic state pruning (DSP) mechanism to reduce the rapidly expanding decision space in large-scale periodic scheduling. By removing infeasible actions online according to endurance and periodic constraints, DSP incorporates physical feasibility directly into the decoding process. This reduces invalid exploration and improves both operational reliability and training efficiency.

(2) We introduce a topology-aware attention (TAA) module to capture the spatial and temporal dependencies among heterogeneous inspection targets. By embedding the structural information of the inspection network into the policy representation, TAA enables the

agent to generate more effective global routing decisions and improves its adaptability to different grid topologies.

(3) We design a constraint-driven composite reward mechanism for long-horizon periodic scheduling and introduce flight efficiency (FE) as an indicator of UAV operational utilization. By balancing immediate routing benefits with long-term periodic and feasibility requirements, this design mitigates myopic optimization behavior and promotes more stable scheduling performance.

The structure of this paper is organized as follows. In Section 2, we review the research progress in related fields. In Section 3, we detail the problem description and mathematical model. In Section 4, we introduce the deep reinforcement learning framework DPTARL. Section 5 presents the experimental results and analysis. Finally, Section 6 concludes the paper.

2. Literature review

Task allocation and path planning have long been studied independently in UAV research. On the task allocation side, prior work has explored auction-based methods (Bai et al., 2023), quantum genetic algorithms (Liu et al., 2021), game-theoretic formulations (Wang et al., 2022), and intuitionistic fuzzy bilateral matching (Gao, 2025). On the path planning side, meta-heuristics such as particle swarm optimization (Na et al., 2023; Huang et al., 2023) and grey wolf optimization (Zhang et al., 2021), as well as DRL-based planners with improved reward shaping (Wang et al., 2025; Luo et al., 2024; Zhang et al., 2023), have been widely adopted. A few recent studies have attempted joint optimization via reinforcement learning combined with multi-agent coordination or integer programming (Li et al., 2024; Mirza et al., 2023; Zhao et al., 2024; Li et al., 2026). However, treating task allocation and path planning separately can lead to short-sighted decisions and reduced efficiency. Even in existing integrated frameworks, large-scale periodic constraints remain difficult to handle effectively (Wu et al., 2025).

Deep reinforcement learning (DRL) and graph-structured representations have substantially advanced UAV scheduling by enabling policies that learn directly from problem topology. DRL-based trajectory planners combined with nonlinear model predictive control have improved tracking precision under dynamic constraints (Shi et al., 2026), while Graph Convolutional Network (GCN)-based approaches have

been applied to multi-UAV task allocation, transforming binary assignment into continuous optimization with strong generalization across large task sets (Zheng et al., 2023). Decentralized Graph Neural Network (GNN) architectures further encode inter-agent communication constraints directly into the assignment policy, enabling scalable coordination without centralized control (Goarin and Loianno, 2024). A double-level DRL framework (DL-DRL) additionally decomposes large-scale UAV scheduling into hierarchical allocation and routing subproblems solved by dedicated encoder-decoder models (Mao et al., 2025). Recent studies have also addressed the Periodic Vehicle Routing Problem (PVRP) using deep learning to embed temporal visit patterns (Shahbazian et al., 2024). In related combinatorial optimization problems, Transformer-based architectures have also been increasingly adopted because of their strong ability to capture long-range dependencies among nodes. Recent studies have also explored broader intelligent and decentralized paradigms for UAV systems. Large Language Models (LLMs) are being integrated into UAV networks to support zero-shot reasoning and more complex decision-making (Javaid et al., 2024). Meanwhile, federated learning has become an important approach for privacy-preserving and secure multi-UAV coordination without centralized servers (Cheng et al., 2025). These developments suggest that topology-aware encoders combined with DRL provide a promising foundation for large-scale inspection scheduling. However, both hierarchical and PVRP-based models usually operate under idealized settings with soft constraints, making them less effective for enforcing strict physical limits such as UAV endurance and daily sortie capacity.

Multi-Agent Reinforcement Learning (MARL) and Hierarchical Reinforcement Learning (HRL) represent the two most relevant paradigms for long-horizon UAV scheduling. On the MARL side, Betalo et al. (2025) proposed a dynamic charging and path planning framework for UAV-powered rechargeable wireless sensor networks using MARL, jointly optimizing charging schedules and trajectories. Similarly, Betalo et al. (2024) developed a MARL-based task scheduling and resource sharing scheme for Open Radio Access Network-empowered multi-UAV systems, achieving improved throughput through coordinated heterogeneous-agent policies. On the HRL side, hierarchical frameworks that separate high-level mission assignment from low-level motion execution have demonstrated improved training stability for UAV navigation (Liu et al., 2023) and energy-efficient computation offloading (Zhou et al., 2023). However, high-level policies operate in abstract subgoal spaces and do not explicitly account for hard constraints. As a result, low-level policies may fail to execute the generated sub-goals within battery or deadline limits.

In summary, decoupled optimization, MARL, and HRL all exhibit myopic optimization behavior when applied to long-horizon periodic inspection problems. Decoupled methods treat task allocation and route planning separately and therefore fail to capture the coupling between these decisions. MARL methods are often affected by non-stationarity and usually do not enforce periodic feasibility at the individual-agent level. HRL methods rely on abstract sub-goals, but these sub-goals may violate hard endurance constraints during execution. As a result, all three types of methods struggle to maintain reliable performance for heterogeneous periodic inspection tasks over long horizons. Another limitation is that existing methods rarely incorporate network topology and dynamic feasibility directly into the learning process, which weakens exploration efficiency and limits scalability. To address these issues, we propose the DPTARL framework, which integrates a Dynamic State Pruning mechanism (DSP) with a Topology-Aware Assignment module (TAA). The DSP removes infeasible actions at each decision step, while the TAA embeds the inspection network graph into the policy representation. Together, these two modules improve decision quality and maintain periodic feasibility over the entire scheduling horizon within a unified framework.

3. Problem description and model formulation

This section introduces PIPUFN in detail. In Section 3.1, we describe the UAV operating mode, inspection tasks, and the key definitions. Then we formulate the mathematical model in Section 3.2.

3.1. Problem description

In this paper, we model the inspection system under the UAV Fixed Nest Station mode as a directed graph network $G = (V, E)$, where V represents the set of nodes and E represents the set of arcs. The node set V is decomposed into four topologically distinct subsets: the Fixed Nest Station node v_0 , substation nodes v_S , transmission tower nodes v_T , and distribution pole nodes v_D . Each element e_w in the arc set E represents a bidirectional connection, characterized by attributes including flight speed v_{fij} , distance d_{ij} , and flight time t_{ij} . A critical feature of this system is the heterogeneity of inspection periodicity. Specifically, substations require daily inspections ($\rho_{v_S} = 28$), transmission towers weekly inspections ($\rho_{v_T} = 4$), and distribution poles monthly inspections ($\rho_{v_D} = 1$). Consequently, the problem is defined over a planning horizon T corresponding to the maximum period. Under the UAV Fixed Nest Station mode, the maximum UAV battery endurance is defined as $B_{\max}$. Energy consumption calculations throughout this paper are based on the sum of flight time and inspection time (total inspection time). The time required for UAV inspection varies by the type of tasks. For substations is $T(\tau_{v_S})$, for transmission towers is $T(\tau_{v_T})$, and for distribution poles is $T(\tau_{v_D})$.

The UAV can perform multiple inspection tasks in one day. Thus, we define the maximum daily inspection number as $\eta_{\max}$. The daily inspection counts are denoted as $\eta = \{\eta_1, \dots, \eta_{28}\}$ and the path set on the r th day is represented as $P_r = \{path_1, \dots, path_{\eta_r}\}$. To illustrate the

spatiotemporal coupling inherent in the proposed model, Fig. 2 presents a representative scenario consisting of one UAV Fixed Nest Station V_N , two substations $\{V_{S1}, V_{S2}\}$, three distribution poles $\{V_{D1}, V_{D2}, V_{D3}\}$, and four transmission towers $\{V_{T1}, V_{T2}, V_{T3}, V_{T4}\}$. The task allocation results and path trajectories are shown for the period days of 1, 2, and 28. For each day of task execution, the remaining inspection counts for each pole are denoted as $\{r'_{S1}, r'_{S2}, r'_{D1}, r'_{D2}, r'_{D3}, r'_{T1}, r'_{T2}, r'_{T3}, r'_{T4}\}$. The trajectory $\{R_1, R_2, R_3\}$ in the figure represents three inspection trajectories of the UAV for the day. This scenario represents only one feasible solution. While substations must be inspected on a daily basis, the inspection of transmission towers and distribution poles can be scheduled within flexible time windows, which gives rise to a large combinatorial space for task allocation.

To strictly formulate the problem within a tractable mathematical framework, we establish the following fundamental operational assumptions:

Definition 1. In modern power inspection systems, the UAV must return to the UAV Fixed Nest Station for recharging or battery swapping upon completing a sortie. State-of-the-art fast-charging technologies or automated swapping mechanisms can fully replenish the battery within an interval of approximately 60 min. In our periodic scheduling formulation, the primary operational bottleneck is the maximum daily sortie capacity (i.e., $\eta_{\max} = 7$), rather than the absolute continuous time. Even under the maximum workload of 7 sorties, the cumulative flight, inspection, and ground-replenishment time strictly remains well within the 24-hour daily operational window. Consequently, the ground charging duration is naturally absorbed into the inter-sortie intervals and does not constrain the macro-level sequential routing. Therefore, it is mathematically decoupled from the objective optimization process.

Definition 2. The UAV battery discharge profile is assumed to be linear. Consequently, all energy consumption in this paper is quantified and calculated based on time metrics, scaled by a constant power rate ρ .

Justification: While micro-level UAV energy consumption is subject to complex, non-linear aerodynamic factors (e.g., wind resistance, acceleration), adopting a linear discharge assumption is a widely

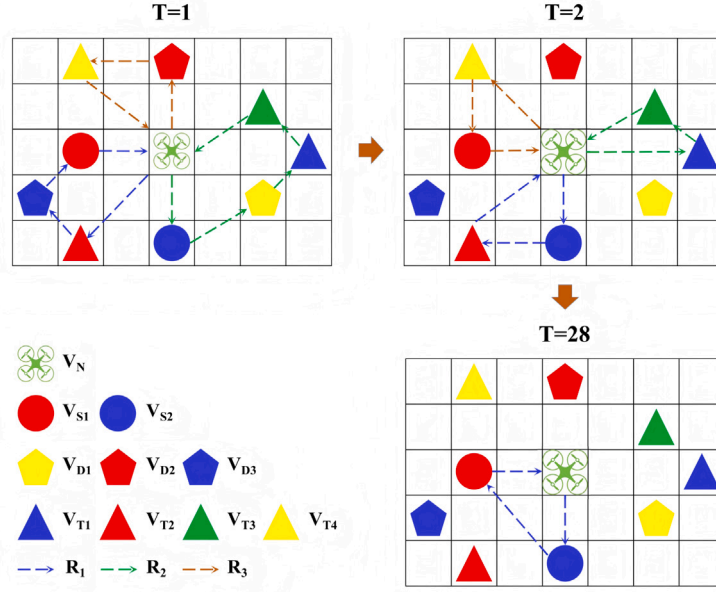


Fig. 2. UAV task allocation and flight trajectories under the UAV Fixed Nest Station mode.

accepted and rigorously validated paradigm in macroscopic, long-horizon scheduling. As demonstrated by Cheng et al. (2020), embedding non-linear aerodynamic dynamics directly into large-scale multi-node routing leads to severe computational intractability. For macro-level scheduling where UAVs maintain a steady cruising speed for the vast majority of the flight, the linear approximation accurately captures the core physical energy boundaries without inducing dimensional explosion, ensuring both mathematical rigor and algorithmic solvability.

Definition 3. The path distance considered in this paper is the two-dimensional Euclidean distance at a constant flight altitude. The vertical movement time required for tower inspection (ascending and descending) is incorporated into the fixed node inspection time $T(\tau_{v_i})$.

3.2. Mixed-integer linear programming formulation

To rigorously quantify the PIPUFN, we formulate PIPUFN as a Mixed-Integer Linear Programming (MILP) model. The objective function in this paper is written as follows:

$$\min \sum_{t \in T} \sum_{(i,j) \in E} \sum_{r \in R_t} d_{ij} x_{ij}^{tr} \quad (1)$$

$$\sum_{j \in V} \sum_{r \in R_t} x_{ij}^{tr} = \sum_{j \in V} \sum_{r \in R_t} x_{ji}^{tr} \quad \forall i \in V, t \in T \quad (2)$$

$$\sum_{j \in V} \sum_{r \in R_t} x_{0j}^{tr} = \sum_{j \in V} \sum_{r \in R_t} x_{j0}^{tr} \quad t \in T \quad (3)$$

$$\sum_{j \in V} x_{0j}^{tr} = 1 \quad t \in T, r \in \eta_t \quad (4)$$

$$\sum_{j \in V} x_{j0}^{tr} = 1 \quad t \in T, r \in \eta_t \quad (5)$$

$$y_i^t \leq \sum_{j \in V} x_{ji}^{tr} \quad \forall i \in V \setminus \{v_0\}, t \in T \quad (6)$$

$$y_i^t = 1 \quad \forall v_i \in V_S, t \in T \quad (7)$$

$$\sum_{i \in W_k} y_i^t \geq 1 \quad v_i \in V_T, W_k = \{7k+1, \dots, 7(k+1)\}, k = 0, \dots, 3 \quad (8)$$

$$\sum_{i \in T} y_i^t \geq 1 \quad \forall v_i \in V_D \quad (9)$$

$$y_i^{t_1} + y_i^{t_2} \leq 1 \quad v_i \in V_T, 0 < t_2 - t_1 < 7 \quad (10)$$

$$u_i^{tr} - u_j^{tr} + |V| x_{ij}^{tr} \leq |V| - 1 \quad \forall i, j \in V \setminus \{v_0\}, i \neq j, t \in T, r \in R_t \quad (11)$$

$$\sum_{(i,j) \in \text{path}_{\eta_t}} \left(\frac{d_{ij}}{v_{fly}} + \tau_j \right) \leq B_{max} \quad \text{path}_{\eta_t} \in P_t \quad (12)$$

$$\eta_t \leq \eta_{max} \quad (13)$$

$$r_i^{t+1} = r_i^t - y_i^t \quad i \in V \setminus \{v_0\}, t \in T \quad (14)$$

The objective function (1) aims to minimize the total path length of all paths in T days. In this model, x_{ij}^{tr} is a binary decision variable, which equals 1 if the UAV flies from node i to node j on the t th day during the r th inspection round, and 0 otherwise. Similarly, y_i^t is a binary variable that takes the value 1 if node i is inspected on the t th day, and 0 otherwise. These variables jointly determine the daily flight routes and inspection schedule of the UAVs. Constraint (2) is a flow balance constraint that ensures that UAV at each node is able to inspect and depart. Constraint (3) shows that the number of UAV depart from the UAV Fixed Nest Station and return to the UAV Fixed Nest Station should be same. Constraints (4)–(5) show that each inspection by UAV requires it to depart from and return to the UAV Fixed Nest Station. Constraint (6) shows the correlation of decision variables. Constraint (7) shows that one inspection is required daily at the substation. Constraint (8) shows that transmission towers require an inspection weekly. Constraint (9) shows that distribution poles need only one inspection during the period. Constraint (10) indicate the limitation of period interval between transmission towers and substations. Constraint (11) is the sub-tour elimination constraint. Constraint (12) indicates that the total inspection time of a single path cannot exceed the UAV battery endurance. Constraint (13) limits the number of daily flights to within the maximum allowable limit. Constraint (14) outlines the update mechanism for the remaining inspection time of nodes. The above constraints (2)–(14) can strictly limit the PIPUFN.

4. A computational framework based on deep reinforcement learning

The formulation in Section 3 shows that the PIPUFN is a combinatorial optimization problem characterized by high-dimensional state spaces and strong spatiotemporal coupling. As the network scale and planning horizon increase, the solution space grows exponentially, making direct solution approaches difficult in practice. Moreover, conventional heuristics often exhibit myopic behavior and fail to account

for dependencies in periodic scheduling. In this paper, we propose a deep reinforcement learning framework, DPTARL, which is built upon Advantage Actor-Critic (A2C) learning paradigm.

4.1. Markov decision process

PIPUFN can be modeled as a Markov decision process $M = (S, \mathcal{A}, P, R, \gamma)$, where S is the state space, $\mathcal{A}$ is the action space, P is the set of locations, $R : S \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, and γ is the discount factor. As this problem involves a long decision horizon, the agent state must be captured in real-time. Thus, the state space is divided into three components: the static tower state space $S^{static} \in \mathbb{R}^{2 \times (n+1)}$, the dynamic tower state space $S^{dynamic} \in \mathbb{N}^{2 \times (n+1)}$ and the UAV state space $C^{UAV} \in \mathbb{R}^3$. The full state can be expressed as $S = (S^{static}, S^{dynamic}, C^{UAV})$.

The concept of moment $\hat{t}$ is introduced in the Markov decision process of this paper. The state at the moment $\hat{t}$ is denoted as $s_{\hat{t}}$ and the action is denoted as $a_{\hat{t}}$ ($a_{\hat{t}} = 0$ represents the UAV returns to the nest and $a_{\hat{t}}$ represents the node i visited by the current action). The position at the moment $\hat{t}$ is denoted as $p_{\hat{t}}$. Since the state space S consists of three parts, the state notation $s_{\hat{t}}$ at moment $\hat{t}$ can be written as $s_{\hat{t}} = (s_{\hat{t}}^{static}, s_{\hat{t}}^{dynamic}, c_{\hat{t}}^{UAV}) \in S$, where $s_{\hat{t}}^{static}$ represents the location information of all nodes and is used as a static state space because the location of nodes are fixed. $s_{\hat{t}}^{dynamic} = [r_{\hat{t}}^{\hat{t}}, l_{\hat{t}}^{\hat{t}}]$ consists of two parts: the number of remaining inspections $r_{\hat{t}}^{\hat{t}}$ at moment $\hat{t}$ and the time of the last inspection $l_{\hat{t}}^{\hat{t}}$. $c_{\hat{t}}^{UAV} = (b_{\hat{t}}, p_{\hat{t}}, d_{\hat{t}})$ represents the real time information of the UAV at moment $\hat{t}$, which consists of three parts: the current power of the UAV $b_{\hat{t}}$, the current position of the UAV $p_{\hat{t}}$ and the current number of day $d_{\hat{t}}$.

4.2. Dynamic state pruning architecture

Given the large solution space of the PIPUFN, we introduce a dynamic state pruning architecture (DSP) to effectively reduce the decision space and improve computational efficiency. The DSP operates by dynamically restricting the set of admissible actions at each decision step based on the current system state. Since each action execution induces a state transition of inspection nodes, the admissible action set is adaptively updated throughout the decision process. Specifically, the DSP is implemented through a binary pruning indicator M_i , where $M_i = 1$ indicates that node i remains admissible for selection and $M_i = 0$ indicates that node i is pruned from the current decision space. The overall pruning indicator is composed of three components: initial mask M_i^{init} , time mask M_i^{time} , and energy mask M_i^{energy} .

$$M_i^{init} = (r_i^0 > 0) \wedge (c_i \neq \text{None}), \quad (15)$$

$$M_i^{time} = M_i^{time1} \vee M_i^{time2} \vee M_i^{time3}, \quad (16)$$

$$M_i^{time1} = l_i^t \neq t \quad \forall v_i \in V_S, \quad (17)$$

$$M_i^{time2} = (l_i^t = -1) \vee (t - l_i^t \geq 7) \quad \forall v_i \in V_T, \quad (18)$$

$$M_i^{time3} = l_i^t \leq 0 \quad \forall v_i \in V_D. \quad (19)$$

Eq. (15) indicates that the number of remaining inspections of node i is greater than 0 and the node is a non-invalid node. The generation of time mask is mainly constrained by the inspection period requirements of substations, transmission towers and distribution poles. Eq. (16) indicates that the time mask of node i needs to satisfy at least one of the cases M_i^{time1} , M_i^{time2} and M_i^{time3} . The time mask constraints for substations, transmission towers, and distribution poles can be expressed as (17)–(19).

$$M_i^{energy} = (E_i^{total} \leq b_{\hat{t}}), \quad (20)$$

$$\Delta t_{move} = \frac{\| (x_{p_{\hat{t}}}, y_{p_{\hat{t}}}) - (x_{p_{\hat{t}+1}}, y_{p_{\hat{t}+1}}) \|}{v_{fly}}, \quad (21)$$

$$\Delta t_{return} = \frac{\| (x_{p_{\hat{t}+1}}, y_{p_{\hat{t}+1}}) - (x_0, y_0) \|}{v_{fly}}, \quad (22)$$

$$E_i^{total} = \rho \cdot (\Delta t_{move} + T(\tau_{a_{\hat{t}}}) + \Delta t_{return}). \quad (23)$$

Eq. (20) indicates that the remaining power of UAV in node i at moment $\hat{t}$. Eq. (21) calculates the time required to constrain the flight from position $p_{\hat{t}}$ to the next position $p_{\hat{t}+1}$. Eq. (22) calculates the time required to fly back to the UAV Fixed Nest Station from position $p_{\hat{t}+1}$. Eq. (23) calculates the total energy E_i^{total} required to ensure that the UAV has enough power to visit node i (position $p_{\hat{t}}$) for inspection and safely return to the Fixed Nest Station. Specifically, the total time duration of this process is scaled by the constant power rate ρ to obtain the equivalent energy consumption, where $T(\tau_{a_{\hat{t}}})$ stands for the required inspection time at node i at moment $\hat{t}$. A special case arises when the sum of all node masks equals 0. In this case, the nest mask is set to 1, allowing the UAV to return to the UAV Fixed Nest Station. This case is expressed as Eq. (24).

$$M_0 = 1, \sum_{i=1}^N M_i = 0. \quad (24)$$

During the inspection process, the feasibility status of each inspection node evolves dynamically as system states change. The proposed DSP updates the admissible decision set accordingly to reflect variations induced by the current system state. By continuously pruning infeasible nodes from the decision space, the DSP substantially reduces the effective solution space and facilitates the generation of feasible action sequences for subsequent policy learning. The pseudocode of the dynamic state pruning architecture is presented as Algorithm 1.

Algorithm 1: Dynamic State Pruning Architecture

```

1 Initialize  $M_i^{init}$ ;
2 for each node  $i \in \{1, \dots, n\}$  do
    // Time constraint verification:
3   if  $v_i \in V_S$  then
4      $M_i^{time} \leftarrow \mathbb{I}[l_i^t \neq \{d_t\}]$ ;
5   end
6   else if  $v_i \in V_T$  then
7      $M_i^{time} \leftarrow \mathbb{I}[l_i^t = -1 \vee (d_t - l_i^t \geq 7)]$ ;
8   end
9   else if  $v_i \in V_D$  then
10     $M_i^{time} \leftarrow \mathbb{I}[l_i^t \leq 0]$ ;
11  end
    // Energy constraint verification:
12   $M_i^{energy} \leftarrow (E_i^{val} \leq E_i)$ ;
    // Action mask:
13   $M_i[i] \leftarrow M_i^{init} \wedge M_i^{time} \wedge M_i^{energy}$ ;
14 end
15 if  $\sum M_i = 0$  then
16    $M_0[0] \leftarrow 1$ ;
17 end
```

4.3. State transfer rules

With the exception of the static state, others need to be updated in real time at the end of each time step $\hat{t}$. To address this situation, this paper designs state transfer rules for problem characteristics. For ease of understanding, the state transfer process is divided into state transfer $f_{nest}(s_{\hat{t}}, a_{\hat{t}})$ in the case of UAV returns to the UAV Fixed Nest Station and state transfer $f_{inspect}(s_{\hat{t}}, a_{\hat{t}})$ when UAV performs inspections. The state transfer expression can be expressed as Eq. (25).

$$s_{\hat{t}+1} = f(s_{\hat{t}}, a_{\hat{t}}) = \begin{cases} f_{nest}(s_{\hat{t}}, a_{\hat{t}}) & a_{\hat{t}} = 0 \\ f_{inspect}(s_{\hat{t}}, a_{\hat{t}}) & a_{\hat{t}} \neq 0. \end{cases} \quad (25)$$

When the UAV returns to the UAV Fixed Nest Station, the battery is fully restored to $B_{\max}$ and the position is updated to the UAV Fixed Nest Station $p_{\hat{\tau}+1} = 0$. If the number of executed missions for the day satisfies $\eta_{\hat{\tau}} \geq \eta_{\max}$, then the date is updated to the next day with $d_{\hat{\tau}+1} = d_{\hat{\tau}} + 1$. Moreover, the number of executed tasks for the day is updated by $\eta_{\hat{\tau}+1} = ((\eta_{\hat{\tau}} + 1) \bmod \eta_{\max}) + 1$. During inspection, the battery level of UAV is updated according to the flight time and the inspection time. The rule for updating the battery level is $b_{\hat{\tau}+1} = \max(b_{\hat{\tau}} - (\Delta t_{\text{move}} + T(\tau_{a_{\hat{\tau}}})) \cdot 0)$. The inspection count for the target node is reduced by $r_{a_{\hat{\tau}}}^{\hat{\tau}+1} = \max(r_{a_{\hat{\tau}}}^{\hat{\tau}} - 1, 0)$, and the date of inspection is recorded as $l_{a_{\hat{\tau}}}^{\hat{\tau}+1} = d_{\hat{\tau}}$. The position of UAV is updated to the currently inspected node if there is still sufficient battery to return $p_{\hat{\tau}+1} = a_{\hat{\tau}}$.

4.4. DPTARL architecture

The DPTARL framework adopts an A2C learning paradigm as its underlying optimization structure. Within this framework, the DPTARL-Actor module is organized as an architectural structure consisting of a Feature Encoder and a topology-aware attention (TAA) decision module.

To process the environmental state information, the encoder is employed to extract compact feature representations. The input tensor is denoted as $X_F \in \mathbb{R}^{B \times C \times L}$, where B denotes the batch size, L represents the sequence length, and C corresponds to the total number of input channels derived from the static, dynamic, and UAV state components. To solve potential dimensional mismatches between the input and output feature representations during deep convolution, a shortcut connection mechanism is introduced, as defined in Eq. (26). The detailed formulations of the associated convolutional operations are provided in Appendix B.

$$X_{\text{shortcut}} = \begin{cases} \text{CovID}(X_F) & \text{if } C \neq H \\ X_F & \text{otherwise.} \end{cases} \quad (26)$$

Based on this architecture, the encoding process handles heterogeneous state information by separately processing the static state s^{static} and the dynamic state s^{dynamic} , which are subsequently fused into a unified embedding tensor E . The corresponding computation steps are defined in Eqs. (27)–(29). The resulting fused feature E provides the input representation for the subsequent decision-making process.

$$E_s = \text{Encoder}(s^{\text{static}}), \quad (27)$$

$$E_d = \text{Encoder}(s^{\text{dynamic}}), \quad (28)$$

$$E = E_s \oplus E_d. \quad (29)$$

Beyond the environmental embedding E , the sequential decision process requires an appropriate initialization of the decoder state. To this end, a dedicated initialization module is introduced to process the learnable parameter tensor $d_{\text{init}} \in \mathbb{R}^{C \times 1}$ and generate the initial decoder hidden state. The corresponding computation is given by $h_0 = \text{Encoder}_{\text{decoder}}(d_{\text{init}})$, where $\text{Encoder}_{\text{decoder}}$ denotes the network module used for decoder state initialization. The detailed computational procedure of this module is provided in Appendix C.

With the state features extracted, the decision-making stage is governed by the interaction between the DSP and the TAA module. As described in Section 4.2, the DSP dynamically delineates the feasible region of the high-dimensional state space. Operating within this feasibility constrained space, TAA serves as a differentiable attention module that models dependency patterns among heterogeneous nodes and maps the resulting attention weights into a stochastic policy. By integrating feasibility information provided by DSP, TAA exploits structural characteristics of the power network and enables the generation of daily task sequences that are both physically feasible and structurally

consistent. The mathematical formulation of the TAA is defined in Eqs. (30)–(33).

$$q = W_q^T h_{\text{dec}}, \quad (30)$$

$$k_i = W_k^T e_i, \quad (31)$$

$$u_i = \mathbf{v}^T \tanh(q + k_i), \quad (32)$$

$$p_i = \frac{e^{u_i}}{\sum_{j=1}^N e^{u_j}} \cdot M_i + (1 - M_i) \cdot (-\infty). \quad (33)$$

In these equations, h_{dec} represents the hidden state of the decoder and h_0 is initial input, e_i represents the fusion feature of node i , $W_q \in \mathbb{R}^{H \times H}$ and $W_k \in \mathbb{R}^{H \times H}$ is the projection matrix, $\mathbf{v} \in \mathbb{R}^H$ is a learnable weight vector used to project hidden layer features into scalar scores, and p_i represents the probability distribution calculated for node i . TAA calculates the score of each node based on h_{dec} and E . The probability distribution of each node is obtained after mask processing and softmax operation. As the decision-making process progresses, the updated h_{dec} and fixed fusion features E are fed into TAA for calculation together. The pseudocode of TAA can be represented as Algorithm 2.

Algorithm 2: Algorithm of Topology-Aware Attention

```

1 Initialize the decoding state:  $h_{\text{dec}} \leftarrow h_0$ ;
2 Feature fusion:
3    $E \leftarrow E_s \oplus E_d$ ;
4 Generate the query vector:
5    $q \leftarrow W_q h_{\text{dec}}$ ;
6 for each node  $i \in \{1, \dots, n\}$  do
7   Generate the key vector:
8      $k_i \leftarrow W_k e_i$ ;
9   Calculate the attention score:
10     $u_i \leftarrow \theta^T \tanh(q + k_i)$ ;
11  Mask processing:
12     $u_i \leftarrow u_i \cdot M_i[i] - \infty \cdot (1 - M_i[i])$ ;
13 end
14 Probability normalization:
15  $p_i \leftarrow \text{softmax}(\{u_i\}_{i=1}^n)$ ;

```

The proposed TAA module is different from standard attention variants. Standard self-attention treats the input as an unstructured sequence. It does not explicitly model physical connectivity. In contrast, TAA embeds the topology of the power grid into its representation. This allows the routing policy to reflect actual physical reachability rather than simple feature similarity. Standard Graph Attention Networks (GATs) usually focus on local one-hop neighborhoods for static feature aggregation. TAA, however, maintains a global receptive field. As a result, the DRL agent can make long-horizon sequential decisions across the entire network while balancing routing efficiency and local topological constraints.

In this paper, the probability of an UAV flying to node i at moment $\hat{\tau}$ can be defined as $\pi_{\hat{\tau}}(a_{\hat{\tau}} | s_{\hat{\tau}}) = TAA(h_{\hat{\tau}}, E)$. Based on the decision process of DPTARL-Actor, the calculation of value estimates is mainly performed in the DPTARL-Critic module. Specifically, the spliced feature is defined as $F = [E_s; E_d]$. Subsequently, a feature compression operation is performed on the basis of this concatenation and global average pooling is applied along the third dimension. The pooling process is defined as $\bar{F} = \frac{1}{L} \sum_{l=1}^L F_{:, :, l}$. Then, the feature $\bar{F}$ is passed through two fully connected layers with a ReLU activation in between. The final value prediction defined as $V_{\theta}(t) = W_2 \cdot \text{ReLU}(W_1 \cdot \bar{F})$.

Moreover, this paper proposes a composite reward mechanism to calculate rewards in every aspect of PIPUFN. The mechanism is evaluated compositely from three dimensions: single-step reward, global reward, and path penalty. Specifically, the single-step reward integrates

three components: base reward, hard penalty, and efficiency penalty. Since the action space of every time step is large, the single-step reward is accumulated in T days. The base reward is defined as $R_{\text{base}} = w_i \cdot n_c$, where w_i is the base reward for inspecting node i and n_c denotes the total number of inspection points completed within T days. To ensure inspection standards, a hard penalty $R_{\text{hard}} = p_{\text{l-hard}} \cdot f_{\text{l-hard}}$ is introduced to penalize transmission towers and distribution poles that are not inspected to requirements, where $p_{\text{l-hard}}$ is the unit penalty for missing inspections of transmission towers during the first week and $f_{\text{l-hard}}$ is the frequency of such occurrences. Additionally, the efficiency reward is designed to penalize redundant flights and imbalanced task distributions. The efficiency reward is defined as $R_{\text{eff}} = \sum_{i=1}^n \sum_{t=1}^T \sum_{m=1}^{n_{\text{max}}} (p_{\text{l-eff}} \cdot f_m^t) + p_{2\text{-eff}} \cdot f_{\text{std}}$, where $p_{\text{l-eff}}$ is the unit penalty for redundant flights on a single day, f_m^t is the number of redundant flights on the t th day during the m th flight, $p_{2\text{-eff}}$ is the unit penalty for global task fluctuation, and f_{std} represents the standard deviation of the number of flights per day over the T days. To ensure full inspection, a global reward $R_{\text{global}} = \frac{n_c}{n} \cdot p_{\text{global}}$ is also introduced, where n is the total number of nodes and p_{global} is the unit reward assigned when inspections are completed as required. Finally, the overall reward combines the above components and penalizes path length. The total reward is defined as $\mathcal{R} = \alpha R_{\text{base}} - \beta R_{\text{hard}} - \kappa R_{\text{eff}} + \delta R_{\text{global}} - \xi D_{\text{total-distance}}$, where $D_{\text{total-distance}}$ is the total path length traveled by the UAV. The process of calculating the mean of the advantage values in our model is defined as (34)–(37).

$$r_t = R_{\text{base}_t} - R_{\text{hard}_t} - R_{\text{eff}_t}, \quad (34)$$

$$G_{t+k} = \sum_{i \in T} \gamma^k r_i \quad t \in T, \quad (35)$$

$$A_t = G_T - V, \quad (36)$$

$$\tilde{A}_t = \frac{A_t - \bar{A}_t}{\sigma_A + \vartheta}. \quad (37)$$

In Eq. (34), r_t denotes the sum of single-step rewards earned on the t th day, which is composed of three components: R_{base_t} is the total basic reward, R_{hard_t} is the hard penalty, and R_{eff_t} is the efficiency penalty obtained on the t th day. Eq. (35) defines the discounted reward G_{t+k} , where γ represents the discount factor. Meanwhile, A_t in Eq. (36) represents the advantage value on the t th day, defined by the difference between G_T and the value estimate V . Eq. (37) further defines the standardized advantage value $\tilde{A}_t$, where $\bar{A}_t$ is the mean of the advantage values, σ_A denotes their variance, and ϑ is a small numerical value.

In the process of calculation, the long discount sequence ignores the rewards in the early and middle stages. In our framework, the DPTARL-Actor calculates task sequences generated by time steps. We calculate daily rewards and discount rewards over any number of days when computing discounted rewards. And the DPTARL-Critic calculates advantage values for any number of days. The loss of DPTARL are defined as (38)–(40).

$$\mathcal{L}_{\text{actor}} = -\mathbb{E} [\tilde{A}(t) \cdot \log \pi_{\theta}(a_t | s_t)], \quad (38)$$

$$\mathcal{L}_{\text{critic}} = \mathbb{E} \left[(V_{\phi}(t) - G_t)^2 \right], \quad (39)$$

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{actor}} + \beta \cdot \mathcal{L}_{\text{critic}}, \quad (40)$$

where $\log \pi_{\theta}(a_t | s_t)$ denotes the log probability of actions output by the DPTARL-Actor network, $\mathcal{L}_{\text{actor}}$ represents the loss of the DPTARL-Actor, $\mathcal{L}_{\text{critic}}$ stands for the loss of the DPTARL-Critic, and $\mathcal{L}_{\text{total}}$ is the total loss of the DPTARL. Besides α and β are the weight coefficients. The training algorithm of DPTARL can be represented as Algorithm 3 and the framework diagram of DPTARL is shown in Fig. 3.

5. Simulation results

In Section 5, we conduct an empirical evaluation of the proposed DPTARL framework using real-world topological data from a regional power network. The performance of different methods is assessed from three key perspectives: total path length (Length), computation time

Table 1

Training parameters setup.

Parameters	Values
Batch size	32
Static size	2
Dynamic size	2
Hidden size	128
Steps	2.0×10^5
γ	0.98
Active function	ReLU
Learning rate	3×10^{-5}

(CT), and flight efficiency (FE). In this work, flight efficiency is defined as the ratio of the total path length to the total inspection time (i.e., the sum of flight time and tower inspection time), serving as a measure of UAV inspection utilization and an evaluation indicator for myopic optimization behavior. For comparative evaluation, the Adaptive Large Neighborhood Search (ALNS) algorithm is selected as a representative heuristic baseline due to its effectiveness in large-scale combinatorial optimization problems. In addition, Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) are included as representative deep reinforcement learning benchmarks. Finally, an ablation study is conducted to quantify the contributions of DSP and TAA modules in the DPTARL.

5.1. Parameter settings

In Section 5.1, we describe the experimental configuration to ensure reproducibility. The simulation environment is implemented in Python, with PyTorch used for neural network construction and training. The topological dataset is derived from real-world geographical coordinates of power infrastructure. All training and testing processes are conducted on a Windows 10 operating system equipped with an Intel Core i7-13700KF CPU (3.40 GHz), 32 GB RAM, and an NVIDIA GeForce RTX 4060 GPU. For comparative evaluation, a strict fairness protocol is adopted. The baseline deep reinforcement learning methods share the same feature encoder architecture and environmental state definitions as the proposed DPTARL framework. This design ensures that performance differences are attributable to the decision-making strategies rather than discrepancies in feature extraction. All network parameters are optimized using the Adam optimizer, and the detailed training hyperparameters are summarized in Table 1.

5.2. Analysis of training

To empirically evaluate the learning dynamics and convergence behavior of the proposed DPTARL framework, we analyze its training performance in comparison with DQN and PPO. To ensure a fair and controlled comparison, all methods are trained under identical environmental settings and share the same feature encoder architecture. Fig. 4 reports the evolution of the average reward over 2.0×10^5 training steps.

As shown in Fig. 4, the three algorithms exhibit different learning behaviors, reflecting their distinct optimization mechanisms and inductive biases. The value-based DQN baseline converges to the lowest performance level. This limitation can be attributed to the high-dimensional and combinatorial nature of the PIPUFN. In periodic inspection tasks with large discrete action spaces, DQN faces substantial challenges in accurately estimating action-value functions, which often leads to overestimation bias and inefficient exploration. As a result, DQN struggles to capture long-horizon dependencies and fails to discover effective scheduling policies. As a policy gradient method, PPO demonstrates a rapid initial performance improvement. However, the learning curve of PPO indicates premature convergence. Although PPO addresses some instability issues found in value-based methods, it lacks explicit mechanisms to account for the topological relationships between inspection targets, which are important for this problem.

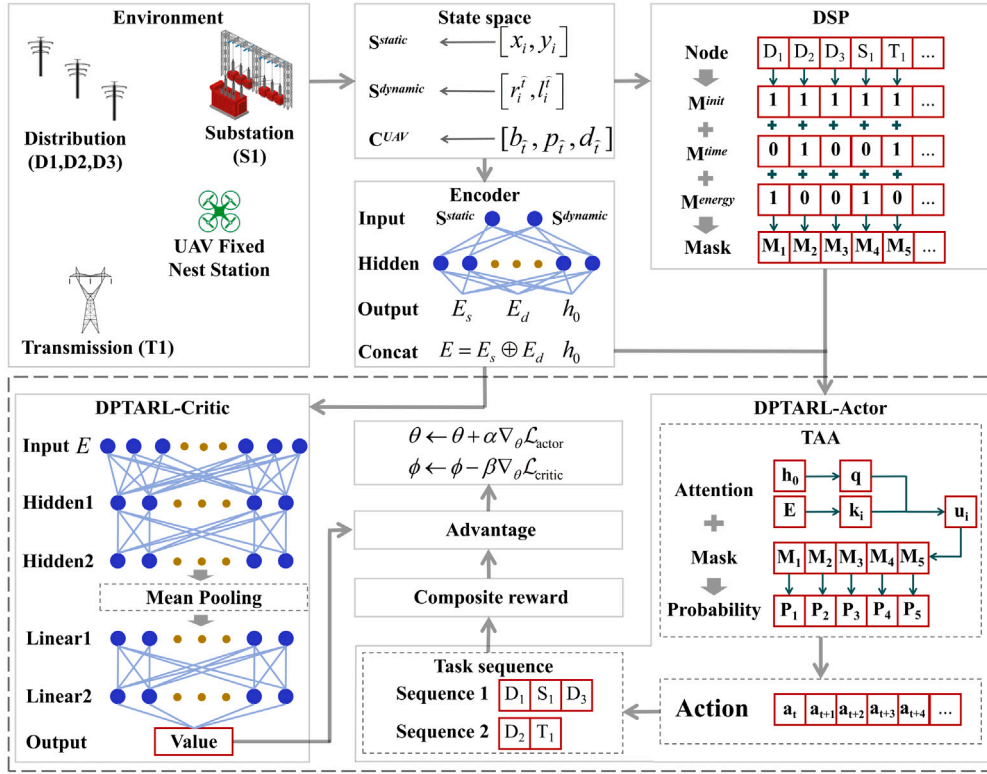


Fig. 3. DPTARL training framework.

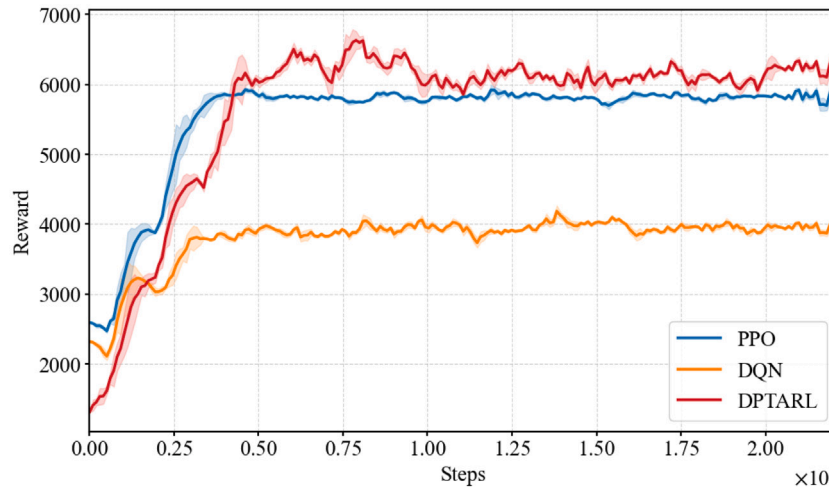


Fig. 4. Average reward of training.

Consequently, the learned policy tends to focus on locally favorable decisions and exhibits limited capability in resolving the complex spatiotemporal couplings required for long-horizon periodic scheduling. In contrast, the proposed DPTARL exhibits a more robust and sustained learning trajectory. While its early stage exploration behavior is comparable to that of the baseline methods, DPTARL achieves a clear performance breakthrough at approximately 4×10^4 training steps and consistently surpasses PPO thereafter, converging to a significantly higher reward level. This progressive overtaking behavior highlights the effectiveness of the proposed architectural design. Specifically, the DSP restricts exploration to a feasibility decision space, thereby

reducing ineffective or invalid actions, while the TAA module enables the policy to capture structural dependencies among heterogeneous nodes. The synergy of these components allows DPTARL to escape suboptimal local solutions and achieve more stable and higher quality scheduling performance.

5.3. Small-scale exact solver verification

To evaluate the solution quality of the proposed DPTARL framework and assess the necessity of using a DRL-based approach, we conduct a comparative experiment against an exact mathematical solver. The

Table 2
Comparison of DPTARL and the exact solver (Gurobi) on small-scale instances.

Size ($ N $)	Gurobi length (km)	DPTARL length (km)	Gap (%)	Gurobi FE	DPTARL FE	Gain (%)	Gurobi time (s)	DPTARL time (s)
5	144.35	152.50	5.64	4.27E-3	4.74E-3	11.01	128.5	1.78
10	168.45	179.30	6.44	4.05E-3	4.72E-3	16.54	>3600 ^a	2.13
15	186.20	201.50	8.22	3.82E-3	4.65E-3	21.73	>3600 ^a	2.61
20	205.60	224.80	9.34	3.55E-3	4.60E-3	29.58	>3600 ^a	3.09

^a Gurobi reached the predefined time limit of 3600 s.

MILP model of the PIPUFN is implemented in the Gurobi optimizer. Because of the NP-hard nature of the problem, the solver time limit is set to 3600 s. To account for the stochasticity of the reinforcement learning policy, the DPTARL results are reported using the average performance over 10 independent runs. The comparison is conducted on small-scale instances with 5 to 20 uniformly distributed nodes, as summarized in Table 2.

The results show a clear difference in computational scalability. For the 5-node instance, Gurobi finds the optimal solution in 128.5 s. However, when the number of nodes increases to 10, the spatiotemporal coupling substantially increases the computational burden, and Gurobi reaches the 3600 s time limit without proving global optimality. In contrast, DPTARL maintains a stable inference time ranging from 1.78 s to 3.09 s. In terms of solution quality, DPTARL shows competitive performance on small-scale instances. For the 5-node instance, the gap between DPTARL and the optimal solution is 5.64%. When the problem size increases to 20 nodes, the gap between DPTARL and the incumbent solution returned by Gurobi within the one-hour limit is 9.34%. A further difference can be observed in terms of flight efficiency (FE). Although Gurobi returns incumbent solutions with shorter path lengths before timing out, the corresponding flight efficiency decreases from 0.00427 to 0.00355 as the problem size increases. By contrast, DPTARL consistently maintains a higher FE, remaining above 0.00460 across all tested scales. These results indicate that DPTARL provides a favorable balance between solution quality and computational efficiency, and can generate operationally effective schedules within seconds.

Computational Complexity Analysis: To formally justify the high inference efficiency demonstrated in the experiments, we analyze the theoretical time complexity of the proposed DPTARL framework. During the inference phase, the computational overhead is primarily dominated by the Topology-Aware Attention (TAA) module and the Dynamic State Pruning (DSP) module. For an instance with N nodes and a hidden feature dimension d , the self-attention mechanism in TAA computes the query-key dot products across all nodes, which yields a complexity of $\mathcal{O}(N^2 \cdot d)$. Subsequently, the DSP module constructs the feasibility mask by evaluating the physical constraints of the available nodes, requiring a linear complexity of $\mathcal{O}(N)$ per decoding step. Across the entire trajectory generation, the overall asymptotic time complexity of DPTARL is bounded by $\mathcal{O}(N^2 \cdot d)$. Compared to the exponential time complexity $\mathcal{O}(2^N)$ typical of exact MILP solvers (e.g., Gurobi), the polynomial complexity of DPTARL mathematically guarantees its superior scalability and millisecond-level real-time applicability in large-scale PIPUFN scenarios.

5.4. Comparative evaluation

In this section, we conduct a comprehensive comparative evaluation of the proposed DPTARL framework against a heuristic baseline ALNS and two representative deep reinforcement learning baselines DQN and PPO. Experiments are performed on four test sets (70, 80, 90, and 100 inspection tasks). The evaluation focuses on three aspects: Length, FE, and CT, which together provide a comprehensive characterization of algorithm performance. To facilitate a quantitative comparison, Table 3 reports the relative performance differences of the baseline methods with respect to DPTARL, computed as $(V_{\text{baseline}} - V_{\text{DPTARL}})/V_{\text{DPTARL}} \times 100\%$.

The average performance across different task scales is illustrated in Fig. 5, while the relative performance gaps between each baseline and DPTARL are quantitatively summarized in Table 3. As shown in Fig. 5(a), DPTARL consistently achieves competitive or shorter path length across all task scales. According to Table 3, DQN produces routes that are on average 8.03% longer than those generated by DPTARL, whereas PPO exhibits a smaller gap of 0.96%. In contrast, the path length obtained by ALNS is nearly identical to that of DPTARL, with an average difference of -0.01% , indicating that both methods exhibit strong spatial optimization capability.

Fig. 5(b) shows more pronounced differences in flight efficiency. According to Table 3, ALNS and DQN exhibit average efficiency reductions of 19.89% and 14.54% relative to DPTARL, respectively, while PPO remains 1.93% lower. This suggests that minimizing path length alone does not guarantee efficient utilization of UAV flight time. DPTARL maintains higher operational efficiency across all task scales, reflecting a better balance between spatial routing and temporal feasibility. Fig. 5(c) reveals a substantial disparity in computation time. ALNS incurs a significantly higher computational cost, with an average time increase of 176.45% compared to DPTARL, and this gap widens as the problem scale increases. In contrast, the computation time of DPTARL remains close to that of PPO and DQN. Although PPO is marginally faster than DPTARL on average (-2.37%), the time difference is small relative to the observed gains in solution quality and efficiency.

As shown in Figs. 6(a) and 6(b), DQN and PPO exhibit wider interquartile ranges and more dispersed outliers, indicating higher variability across independent runs. This instability is particularly evident in flight efficiency, where learning-based baselines show noticeable fluctuations. In contrast, DPTARL demonstrates a compact distribution. The computation time distributions in Fig. 6(c) further illustrate the computational characteristics of different methods. ALNS consistently incurs the highest computation time, reflecting its limited scalability in large-scale instances. In contrast, DQN and PPO exhibit lower average computation time with very narrow interquartile ranges, indicating highly stable inference efficiency. DPTARL shows a moderate computation time that is higher than that of PPO and DQN, accompanied by a wider but still controlled dispersion. This behavior suggests that the additional structural reasoning introduced by the DSP and TAA modules incurs a certain computational overhead, while maintaining a predictable and bounded inference cost. In practical scenarios, this moderate increase in computation time is compensated by substantial gains in solution quality and flight efficiency, as demonstrated in Figs. 5 and 7. In Fig. 7(a), DPTARL solutions cluster in the lower-left region, indicating length that are relatively short while requiring moderate computation time. In contrast, ALNS solutions are mainly distributed in the upper region due to their high computational cost, whereas DQN and PPO tend to spread toward the right, corresponding to longer path length. A similar trend is observed in Fig. 7(b), DPTARL occupies the upper-left region, achieving higher flight efficiency with comparatively shorter length. By comparison, DQN and ALNS exhibit lower flight efficiency under similar path length conditions. In Fig. 7(c), DPTARL is located in a region characterized by relatively high flight efficiency with bounded computation time, indicating a favorable balance between operational efficiency and computational overhead.

Combining the quantitative results with the visual evidence, it can be observed that DPTARL achieves a favorable balance across all

Table 3
Quantitative comparison.

Tasks	ALNS vs. DPTARL			DQN vs. DPTARL			PPO vs. DPTARL		
	Length	FE	CT	Length	FE	CT	Length	FE	CT
100	2.99%	-19.00%	171.53%	9.84%	-14.28%	-13.23%	4.12%	-2.28%	-25.33%
90	1.08%	-19.21%	172.55%	7.55%	-14.34%	26.27%	1.11%	-2.48%	11.12%
80	-2.14%	-20.50%	201.26%	7.25%	-14.66%	14.70%	1.62%	-1.73%	3.50%
70	-1.97%	-20.85%	160.46%	7.48%	-14.86%	15.24%	0.23%	-1.22%	1.25%
Average	-0.01%	-19.89%	176.45%	8.03%	-14.54%	10.75%	0.96%	-1.93%	-2.37%

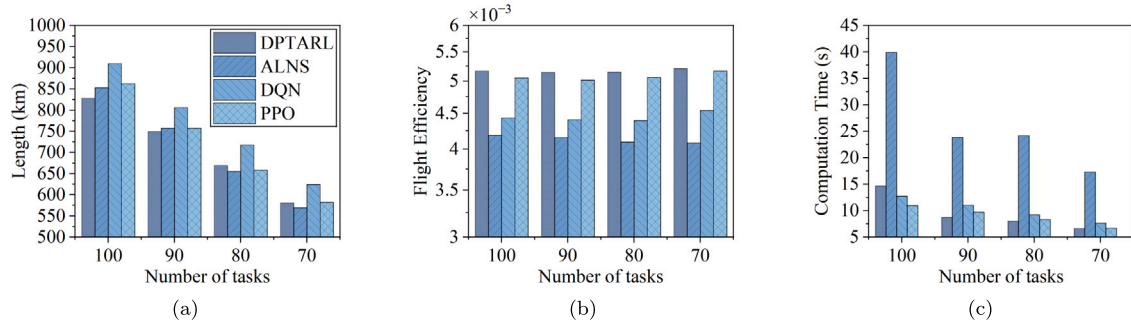


Fig. 5. Comparison of macro performance metrics across different task scales.

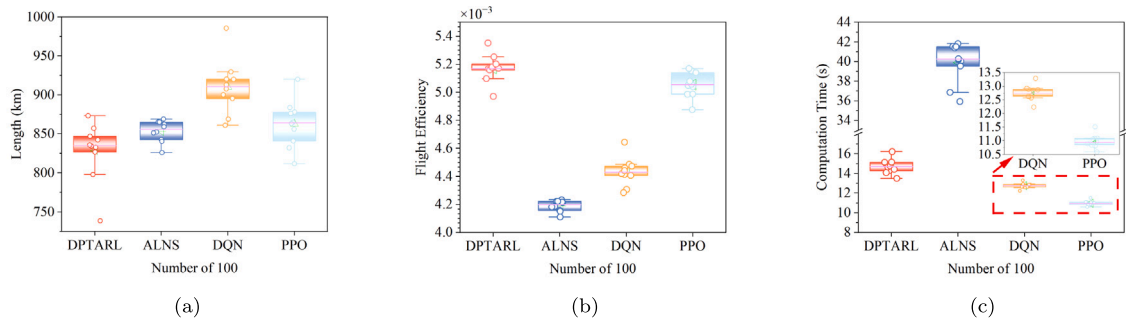


Fig. 6. Statistical distribution and robustness comparison.

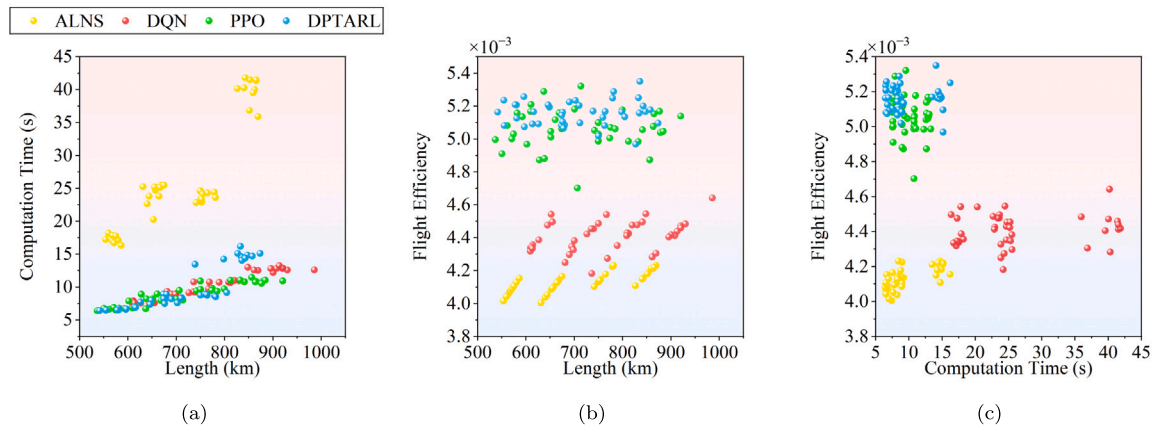


Fig. 7. Multi objective trade-off visualization.

three evaluation dimensions. While ALNS is competitive in terms of path length, it suffers from excessive computation time and reduced operational efficiency. Standard DRL methods offer faster inference but exhibit inferior solution quality. By contrast, DPTARL consistently delivers high-quality, efficient, and stable solutions across different task scales, demonstrating its suitability for PIPUFN.

Algorithm 3: Training Algorithm of DPTARL

```

1 Initialize the policy network parameters  $\pi_\theta$  of DPTARL-Actor,
  the value network parameters  $V_\phi$  of DPTARL-Actor and the
  experience buffer  $\Omega$ ;
2 for  $episode = 1, \dots, N_{ep}$  do
3   Reset the environment to obtain the initial state
      $s_0 = (s_0^{route}, s_0^{time}, s_0^{energy})$ ;
4   while the terminal state is not reached do
5     Generate the action mask  $M_t$  (Algorithm 1);
6     Obtain the action probability distribution  $P_t$  through
       the TAA (Algorithm 2);
7     Sample an action:  $a_t \sim \text{Categorical}(P_t)$ ;
8     Execute action  $a_t$  to get  $s_{t+1}, r_t = f(s_t, a_t)$ ;
9     Calculate the single-step immediate reward  $r_t$ ;
10    Store  $(s_t, a_t, r_t, s_{t+1})$  into  $\Omega$ ;
11    Complete the period tasks;
12  end
13  Calculate the value prediction:  $V_\phi(t)$ ;
14  Calculate the discount reward:  $G$ ;
15  Calculate the advantage function:  $A \in \tilde{A}_t$ ;
16  Update the policy network:
17     $\nabla_\theta \mathcal{L}_{actor} = \mathbb{E}[A \cdot \log \pi_\theta(a_t | s_t)]$ ;
18  Update the value network:
19     $\nabla_\phi \mathcal{L}_{critic} = \mathbb{E}[(V_\phi(t) - G)^2]$ ;
20  Policy gradient update:
21     $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}_{actor}, \quad \phi \leftarrow \phi - \beta \nabla_\phi \mathcal{L}_{critic}$ 
22 end

```

5.5. Ablation experiment

To systematically examine the individual and joint contributions of the proposed components, we conduct an extended ablation study. In the DPTARL framework, DSP and TAA are integrated into a unified architecture. To quantify their effects, we evaluate four variants:

- (1) **A2C**, a baseline without DSP and TAA modules.
- (2) **NO-DSP**, DPTARL without the DSP module.
- (3) **NO-TAA**, DPTARL without the TAA module.
- (4) **DPTARL**, the complete proposed framework.

All variants are evaluated over 10 independent runs. The quantitative results are summarized in Table 4, which presents the three performance dimensions clearly.

As shown in Fig. 8(a), DPTARL consistently achieves the shortest path length. Notably, the NO-TAA variant shows a clear degradation in spatial optimization, and its path length approaches that of the vanilla A2C baseline. This result indicates that DSP helps restrict the agent to feasible actions, whereas TAA improves its ability to capture structural relationships and construct compact inspection sequences.

In terms of FE, Fig. 8(b) provides further evidence on the mitigation of myopic optimization behavior. In this study, FE is used as a practical indicator of myopic optimization behavior. A higher FE suggests that the agent can better balance immediate rewards with long-term operational endurance. As shown in Table 4, DPTARL achieves the highest FE, which indicates a stronger ability to alleviate myopic decision tendencies. In contrast, the NO-DSP variant shows a substantial drop in efficiency and performs even worse than A2C. This result suggests that removing the physical feasibility boundaries imposed by DSP makes the agent more likely to pursue immediately reachable nodes,

Table 4

Average ablation results across four model variants.

Model	Length (km)	FE	CT (s)
A2C	993.88	4.705E−3	15.26
NO-DSP	921.90	4.375E−3	14.42
NO-TAA	971.68	4.509E−3	13.99
DPTARL	828.52	5.171E−3	14.69

Table 5

Sensitivity analysis of penalty multipliers.

Penalty multiplier	Length (km)	FE	CT (s)
Under-penalized (0.5×)	913.10	4.513E−3	18.15
DPTARL (1.0×)	828.52	5.174E−3	14.69
Over-penalized (2.0×)	872.97	4.416E−3	14.53

which increases short-sighted decisions and weakens overall scheduling quality.

Table 4 and Fig. 8(c) show that introducing either DSP or TAA alone reduces computation time. Although the DPTARL framework is slightly slower than the single-module variants, it still maintains a stable inference time. More importantly, this modest time cost is accompanied by clear improvements in path length and flight efficiency. Therefore, the combined use of DSP and TAA provides a more favorable balance between decision quality and computational efficiency.

Ablation results show that DSP and TAA play complementary roles in the proposed framework. DSP improves action feasibility and stabilizes long-horizon scheduling, whereas TAA enhances structural perception and sequence optimization. In DPTARL, the two modules deliver the best overall performance across length, flight efficiency, and computation time. This confirms the effectiveness of DPTARL.

5.6. Sensitivity analysis

To evaluate the robustness of the proposed composite reward mechanism and justify the hyperparameter settings, we conduct a sensitivity analysis on the penalty multipliers. In DPTARL, the hard penalty and the efficiency penalty play an important role in model training. We examine the effect of these penalty weights by comparing the base DPTARL model (1.0× multiplier) with two extreme settings: an under-penalized variant (0.5×) and an over-penalized variant (2.0×).

The average performance on the three core metrics is summarized in Table 5. The results show that the base DPTARL model (1.0×) provides the best overall balance, achieving the shortest path length and the highest flight efficiency. When the penalty multiplier is reduced to 0.5×, the length increases substantially to 913.10 km, and the flight efficiency decreases to 4.513×10^{-3} . This degradation suggests that a weak penalty signal is insufficient to constrain the agent effectively. As a result, the agent is more likely to favor immediate local rewards, leading to redundant flights and poorer global routing efficiency. The weaker restriction also enlarges the effective search space, which further increases inference time. When the penalty multiplier is increased to 2.0×, the length rises to 872.97 km, and the flight efficiency drops to the lowest value among all settings (4.416×10^{-3}). In this over-penalized setting, the agent becomes overly conservative. To avoid large penalties, it tends to prioritize immediate constraint satisfaction over path quality, which leads to inefficient flight patterns such as zigzagging routes or premature returns to the UAV Fixed Nest Station.

Overall, these results indicate that the 1.0× penalty setting in DPTARL achieves the most favorable balance between physical constraint enforcement and global routing efficiency. This finding supports the rationality of the reward design and shows that appropriate penalty scaling is important for mitigating short-sighted decisions in long-horizon scheduling.

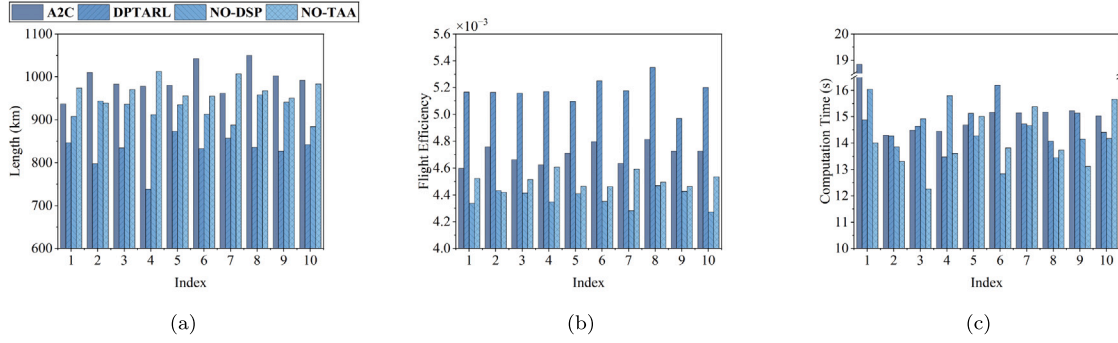


Fig. 8. Result of ablation experiment.

6. Conclusion

This paper studies the highly constrained PIPUFN problem and presents the DPTARL framework. The proposed framework addresses the computational limitations of exact solvers and reduces the myopic optimization tendencies of traditional DRL methods. The main strength of the framework lies in the joint use of DSP and TAA. DSP enforces physical feasibility by pruning invalid actions. TAA then captures global topological dependencies within the resulting feasible decision space. In this way, the framework supports long-horizon routing decisions and reduces reliance on manually designed soft penalties. Extensive experiments demonstrate the effectiveness of the proposed framework. In small-scale verification, DPTARL achieves near-optimal performance, with an optimality gap below 10%, while providing substantial inference acceleration over the exact MILP solver. In large-scale evaluations against representative baselines, including PPO, DQN, and ALNS, DPTARL also shows clear performance gains. It reduces the average path length by 16.64% and improves flight efficiency by 9.89%. In addition, the visualization of the 28-day continuous UAV flight trajectories and the aggregated spatial coverage heatmap (see Figs. D.9 and D.10) collectively provide further evidence of the engineering feasibility. These visual results confirm that the proposed DPTARL framework can successfully execute precise micro-level routing while consistently satisfying all heterogeneous periodic requirements throughout the long-horizon operation. Beyond power inspection, this work provides a generalizable perspective on applying deep reinforcement learning to large-scale, periodic, and long-horizon scheduling problems.

CRedit authorship contribution statement

Yuxuan Sun: Writing – review & editing, Writing – original draft, Visualization, Methodology, Formal analysis, Conceptualization. **Feng Gao:** Supervision, Methodology. **Yongjie Xue:** Validation, Supervision. **Bin Yu:** Conceptualization.

Funding

This work was supported by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM104) and the National Natural Science Foundation of China (Grant No. 52232011).

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Bin Yu reports financial support was provided by National Natural Science Foundation of China (52232011 and 52225209). If there are other

authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Notations used in this paper

The notations used throughout the model formulation and algorithm design are listed in Table A.6.

Appendix B. Computational process of convolution operation

The steps in the Encoder begin with the convolution operation defined as (B.1)–(B.3).

$$H_{Con} = \text{ConvID}(X_F) = W_1 \cdot X_F + b_1, \quad (\text{B.1})$$

where $W_1 \in R^{H \times C \times L}$ represents the weight matrix, $b_1 \in R^H$ represents the bias vector, and H represents the number of output channels. Next, the batch normalization operation is defined as:

$$H_{BN} = \text{BN}(H_{Con}) = \chi_1 \frac{H_{Con} - \mu_1}{\sqrt{\sigma_1^2 + \epsilon}} + \beta_1, \quad (\text{B.2})$$

where χ_1, β_1 are learnable parameters; μ_1, σ_1^2 are the mean and variance of the current batch and ϵ is a very small constant. Finally ReLU operation is performed on the normalized result defined as:

$$Y = \text{ReLU}(H_{BN}) = \max(0, H_{BN}). \quad (\text{B.3})$$

Appendix C. Computational process of Encoder_{decoder}

The computational process of Encoder_{decoder} can be represented as (C.1)–(C.4).

$$X_F = \begin{cases} X_{in} & \text{if } X_{in} \in (B, C, 1), \\ X_{in} \cdot \text{permute}(0, 2, 1) & \text{otherwise,} \end{cases} \quad (\text{C.1})$$

$$H_1 = \text{ReLU}(\text{BN}(\text{Conv1D}(X_F))), \quad (\text{C.2})$$

$$H_2 = \text{BN}(\text{Conv1D}(H_1)), \quad (\text{C.3})$$

$$h_0 = \text{ReLU}(H_2 + X_{\text{shortcut}}) \in R^{B \times H \times A}. \quad (\text{C.4})$$

Appendix D. Visualization of global inspection coverage

In the considered PIPUFN scenario, the UAV can execute up to seven sorties per day over a 100-node network within a 28-day scheduling horizon. To intuitively demonstrate the effectiveness of the proposed path planning method, we first visualize the actual UAV trajectories over the entire period, as shown in Fig. D.9. This visualization directly presents the spatial routing decisions generated by the DPTARL

Table A.6

Notations.

Notation	Definition
V	Set of nodes, where $V = V_S \cup V_T \cup V_D \cup v_0$.
V_S	Set of substation nodes.
V_T	Set of transmission nodes.
V_D	Set of distribution nodes.
Pos	Set of node coordinate, where $Pos = \{X, Y\}$.
X	Set of horizontal coordinates.
Y	Set of vertical coordinates.
E	Set of arcs.
G	Set of graph structure.
η	Set of daily inspections.
P_t	Set of path on the t th day.
T	Set of cycle days, where $T = \{1, 2, \dots, 28\}$.
M	Markov decision process.
S	Set of state-space.
A	Set of action-space.
P	Set of position-space.
R	Set of rewards.
v_i	Power nodes to be inspected, where $v_i \in V$.
v_0	UAV Fixed Nest Station node.
ρ_v	Total number of inspection requirements for each type of node, where $v_i \in V$.
r_i^r	Remaining number of required inspections for node i on r th day.
$T(\tau_{v_i})$	Single inspection time consumed of node i , where $T(\tau_{v_s}) = 600$ s, $T(\tau_{v_T}) = 300$ s, $T(\tau_{v_D}) = 60$ s.
v_{fly}	UAV flight speed, where $v_{fly} = 0.01$ km/s.
d_{ij}	Euclidean distance from node i to j .
t_{ij}	Flight time from node i to j .
$B_{\max}$	Maximum UAV battery endurance, where $B_{\max} = 3000$ s.
pos_i	Plane coordinates of node i .
$\eta_{\max}$	Maximum number of flights per day.
η_t	Number of missions carried out on t th day.
γ	Discount factor, where $\gamma \in [0, 1)$.
$s_{\bar{t}}$	Characteristics of the state at moment $\bar{t}$.
$s_{\bar{t}}^{static}$	Characteristics of static state.
$s_{\bar{t}}^{dynamic}$	Characteristics of the dynamic state at moment $\bar{t}$.
$c_{\bar{t}}^{UAV}$	Characteristics of the UAV state at the moment $\bar{t}$.
$b_{\bar{t}}$	Remaining power of the UAV at moment $\bar{t}$.
$p_{\bar{t}}$	Position of the UAV at moment $\bar{t}$.
$d_{\bar{t}}$	Number of days at moment $\bar{t}$.
$r_i^{\bar{t}}$	Remaining number of required inspections for node i at moment $\bar{t}$.
$l_i^{\bar{t}}$	Inspection date of node i at moment $\bar{t}$.
$a_{\bar{t}}$	Action at moment $\bar{t}$.
M_i	Action mask for node i .
M_i^{init}	Initial mask for point i .
M_i^{time}	Time constraint mask for point i .
M_i^{energy}	Energy constraint mask for point i .
M_0	1, if UAV forced return to UAV Fixed Nest Station.
f_{nest}	State transfer rules for UAV returns to UAV Fixed Nest Station.
f_{inspect}	State transfer rules during inspections.
Δt_{return}	Time of UAV returns to UAV Fixed Nest Station from location $p_{\bar{t}+1}$.
Δt_{move}	Time of UAV from $p_{\bar{t}}$ to move to $p_{\bar{t}+1}$.
E_i^{total}	Energy consumption of node i , where $E_i^{total} = \Delta t_{\text{move}} + T(\tau_{a_{\bar{t}}}) + \Delta t_{\text{return}}$.
$T(\tau_{a_{\bar{t}}})$	Detection time required for the node corresponding to action $a_{\bar{t}}$.
X_F	Model input features.
B	Batch size.
C	Number of input channels.
L	Length of sequence.
p_i	Selection probability of node i .
E_s	Static coding.
E_d	Dynamic coding.
E	Fusion coding.
F	Feature splicing results.
$\bar{F}$	Global average pooling results.
$V_{\phi}(t)$	Results of value projections at t th day.
h_{dec}	Hidden state of the decoder, initial decoding state is h_0 .
$D_{\text{total-distance}}$	Total path length.
ξ	Unit penalty of distance.
R_{global}	Global reward.
n	Total number of nodes.
p_{global}	Reward for node unit completing inspections.
R_{eff}	Redundant flight efficiency penalties.

(continued on next page)

Table A.6 (continued).

f_m^t	Redundant flights for on r th day, m th flight.
p_{1-eff}	Unit penalty for single redundant flight in a single day.
p_{2-eff}	Unit penalty for fluctuation in the volume of tasks.
f_{std}	Standard deviation of mission volume fluctuations.
R_{hard}	Hard penalty.
p_{1-hard}	Unit penalty for not inspecting transmission towers in week one.
f_{1-hard}	Number of transmission towers not inspected in the first week.
R_{base}	Base inspection reward.
w_i	Base inspection reward of node i .
n_e	Total number of inspection nodes.
R_{base_r}	Total base reward earned on r th day.
R_{hard_r}	Total hard penalties earned on r th day.
R_{eff_r}	Total efficiency penalty obtained on r th day.
G_t	Discount rewards on r th day.
r_{t+k}	Rewards earned by agent on day $t + k$.
A_t	Advantage value on r th day.
$\bar{A}_t$	Mean value of advantage values.
$\hat{A}_t$	Advantage value after standardization on r th day.
ρ	Constant power rate.
ϑ	A very small constant, where $\vartheta = 10^{-8}$.
σ_{A_t}	Variance of advantage values.
$\mathcal{L}_{actor}$	Loss of DPTARL-Actor.
$\mathcal{L}_{critic}$	Loss of DPTARL-Critic.
$\mathcal{L}_{total}$	Total loss of training.
x_{ij}^{tr}	1, if UAV flies from i to j on the on r th day, the r th round.
y_i^r	1, if node i is inspected on r th day.

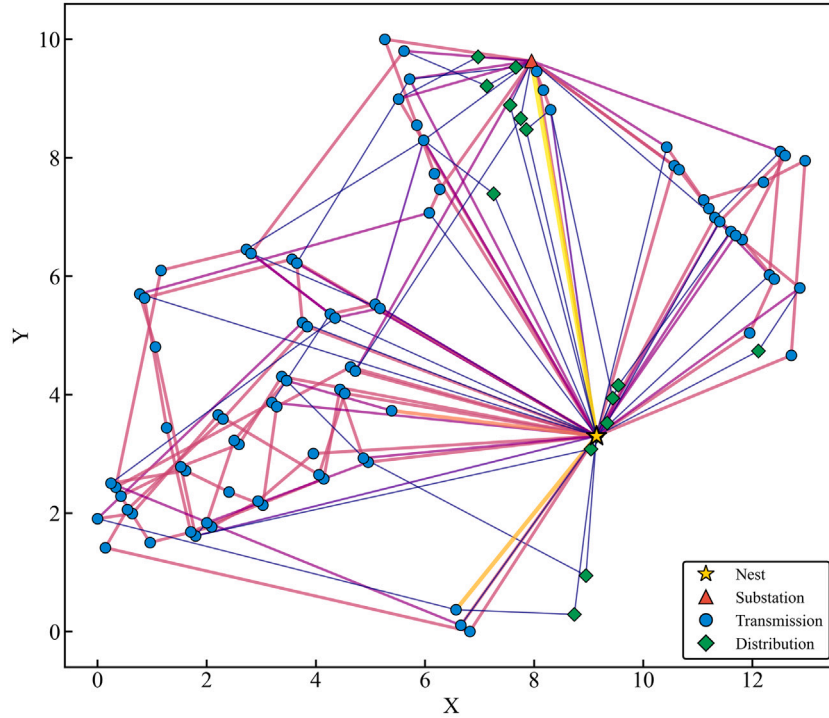


Fig. D.9. Flight trajectories of the 100-node network over the 28-day horizon.

framework. However, because of the high frequency of repeated visits over the long-horizon schedule, the raw trajectory lines inevitably show substantial visual overlap, which makes the overall periodic scheduling pattern difficult to interpret.

To provide a clearer and more intuitive illustration of the global completion of heterogeneous inspection tasks, we further introduce a

continuous spatial coverage heatmap, as shown in Fig. D.10. Instead of displaying dense and overlapping trajectories, this macroscopic view aggregates the cumulative inspection frequencies of all nodes over the 28-day period.

In Fig. D.10, the color gradient from blue to red reflects the spatial intensity of UAV visits. High-frequency targets, such as the central

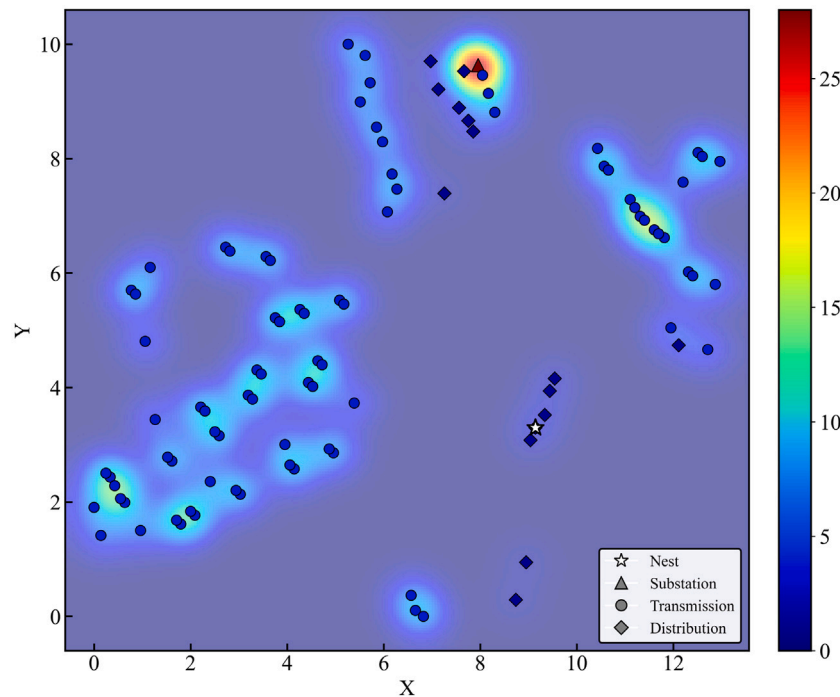


Fig. D.10. Inspection heatmap of the 100-node network over the 28-day horizon.

substation requiring 28 visits, are clearly represented as high-density hot zones. In contrast, the surrounding transmission and distribution nodes exhibit correspondingly lower visitation densities. Taken together, the trajectory plot and the heatmap show that the proposed DPTARL framework can effectively manage the underlying spatiotemporal coupling and satisfy all heterogeneous inspection frequency requirements throughout the scheduling horizon.

Data availability

Data will be made available on request.

References

- Bai, X., Fielbaum, A., Kronmüller, M., Knoedler, L., Alonso-Mora, J., 2023. Group-based distributed auction algorithms for multi-robot task assignment. *IEEE Trans. Autom. Sci. Eng.* 20 (2), 1292–1303. <http://dx.doi.org/10.1109/TASE.2022.3175040>.
- Betalo, M.L., Leng, S., Abishu, H.N., Dharejo, F.A., Seid, A.M., Erbad, A., Naqvi, R.A., Zhou, L., Guizani, M., 2024. Multi-agent deep reinforcement learning-based task scheduling and resource sharing for O-RAN-empowered multi-UAV-assisted wireless sensor networks. *IEEE Trans. Veh. Technol.* 73 (7), 9247–9261. <http://dx.doi.org/10.1109/TVT.2023.3330661>.
- Betalo, M.L., Leng, S., Seid, A.M., Abishu, H.N., Erbad, A., Bai, X., 2025. Dynamic charging and path planning for UAV-powered rechargeable WSNs using multi-agent deep reinforcement learning. *IEEE Trans. Autom. Sci. Eng.* 22, 15610–15626. <http://dx.doi.org/10.1109/TASE.2025.3558945>.
- Caballero, A., Paneque, J.L., Martínez-de Dios, J.R., Maza, I., Ollero, A., 2024. Multi-UAV systems for inspection of industrial and public infrastructures. In: *Infrastructure Robotics: Methodologies, Robotic Systems and Applications*. pp. 285–303. <http://dx.doi.org/10.1002/97811394162871.ch14>.
- Cacace, J., Orozco-Soto, S.M., Suarez, A., Caballero, A., Orsag, M., Bogdan, S., Vasiljevic, G., Ebeid, E., Rodriguez, J.A.A., Ollero, A., 2021. Safe local aerial manipulation for the installation of devices on power lines: AERIAL-CORE first year results and designs. *Appl. Sci.* 11 (13), 6220. <http://dx.doi.org/10.3390/app11136220>.
- Cheng, C., Adulyasak, Y., Rousseau, L.-M., 2020. Drone routing with energy function: Formulation and exact algorithm. *Transp. Res. B* 139, 364–387. <http://dx.doi.org/10.1016/j.trb.2020.06.011>.
- Cheng, Y., Zhang, W., Zhang, Z., Zhang, C., Wang, S., Mao, S., 2025. Toward federated large language models: Motivations, methods, and future directions. *IEEE Commun. Surv. Tutorials* 27 (4), 2733–2764. <http://dx.doi.org/10.1109/COMST.2024.3503680>.
- Gao, F., 2025. A novel unmanned aerial vehicles task allocation approach based on the intuitionistic fuzzy multi-criteria bilateral matching-based decision-making method. *Eng. Appl. Artif. Intell.* 162, 112669. <http://dx.doi.org/10.1016/j.engappai.2025.112669>.
- Gazzea, M., Pacevicius, M., Dammann, D.O., Sapronova, A., Lunde, T.M., Arghandeh, R., 2022. Automated power lines vegetation monitoring using high-resolution satellite imagery. *IEEE Trans. Power Deliv.* 37 (1), 308–316. <http://dx.doi.org/10.1109/TPWRD.2021.3059307>.
- Goarin, M., Loianno, G., 2024. Graph neural network for decentralized multi-robot goal assignment. *IEEE Robot. Autom. Lett.* 9 (5), 4051–4058. <http://dx.doi.org/10.1109/LRA.2024.3371254>.
- Huang, C., Zhou, X., Ran, X., Wang, J., Chen, H., Deng, W., 2023. Adaptive cylinder vector particle swarm optimization with differential evolution for UAV path planning. *Eng. Appl. Artif. Intell.* 121, 105942. <http://dx.doi.org/10.1016/j.engappai.2023.105942>.
- Javaid, S., Fahim, H., He, B., Saeed, N., 2024. Large language models for UAVs: Current state and pathways to the future. *IEEE Open J. Veh. Technol.* 5, 1166–1192. <http://dx.doi.org/10.1109/OJVT.2024.3446799>.
- Li, L., Sang, H., Zhang, D., 2026. An integrated task and path planning approach for intelligent coordination of agricultural robots. *Expert Syst. Appl.* 302, 130602. <http://dx.doi.org/10.1016/j.eswa.2025.130602>.
- Li, Z., Shi, N., Zhao, L., Zhang, M., 2024. Deep reinforcement learning path planning and task allocation for multi-robot collaboration. *Alex. Eng. J.* 109, 408–423. <http://dx.doi.org/10.1016/j.aej.2024.08.102>.
- Liu, Z., Cao, Y., Chen, J., Li, J., 2023. A hierarchical reinforcement learning algorithm based on attention mechanism for UAV autonomous navigation. *IEEE Trans. Intell. Transp. Syst.* 24 (11), 13309–13320. <http://dx.doi.org/10.1109/TITS.2022.3225721>.
- Liu, Y., Shi, J., Liu, Z., Huang, J., Zhou, T., 2019. Two-layer routing for high-voltage powerline inspection by cooperated ground vehicle and drone. *Energies* 12 (7), 1385. <http://dx.doi.org/10.3390/en12071385>.
- Liu, P., Wang, B., Liu, W., Zhang, L., 2021. Multi-task allocation of multi-UAV coalition based on improved quantum genetic algorithm. In: *2021 40th Chinese Control Conference*. CCC, Shanghai, China, pp. 1802–1807. <http://dx.doi.org/10.23919/CCC52363.2021.9550641>.
- Luo, X., Wang, Q., Gong, H., Tang, C., 2024. UAV path planning based on the average TD3 algorithm with prioritized experience replay. *IEEE Access* 12, 38017–38029. <http://dx.doi.org/10.1109/ACCESS.2024.3375083>.
- Mao, X., Wu, G., Fan, M., Cao, Z., Pedrycz, W., 2025. DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV. *IEEE Trans. Autom. Sci. Eng.* 22, 1028–1044. <http://dx.doi.org/10.1109/TASE.2024.3358894>.
- Mbuli, N., Mendu, B., Seitshiro, M., Pretorius, J.-H.C., 2020. What outage attributes affect the frequencies of forced transformer outages in eskom regions? Evidence from 2007–2016 data. *Energy Rep.* 6, 474–482. <http://dx.doi.org/10.1016/j.egy.2020.11.211>.

- Mirza, I.S., Shah, S., Siddiqi, M.Z., Wuttisittikukij, L., Sasithong, P., 2023. Task assignment and path planning of multiple unmanned aerial vehicles using integer linear programming. In: TENCON 2023 - 2023 IEEE Region 10 Conference. TENCON, Chiang Mai, Thailand, pp. 547–551. <http://dx.doi.org/10.1109/TENCON58879.2023.10322503>.
- Na, Y., Li, Y., Chen, D., Yao, Y., Li, T., Liu, H., Wang, K., 2023. Optimal energy consumption path planning for unmanned aerial vehicles based on improved particle swarm optimization. Sustainability 15 (16), 12101. <http://dx.doi.org/10.3390/su151612101>.
- Shahbazian, R., Pugliese, L.D., Guerriero, F., Macrina, G., 2024. Integrating machine learning into vehicle routing problem: Methods and applications. IEEE Access 12, 93087–93115. <http://dx.doi.org/10.1109/ACCESS.2024.3422479>.
- Shi, J., Zhou, S., Man, X., Li, C., Guo, F., 2026. Trajectory planning and tracking for UAVs with deep reinforcement learning and adaptive nonlinear MPC. Expert Syst. Appl. 303, 130649. <http://dx.doi.org/10.1016/j.eswa.2025.130649>.
- Tedeschi, P., Olieri, G., Di Pietro, R., 2020. Leveraging jamming to help drones complete their mission. IEEE Access 8, 5049–5064. <http://dx.doi.org/10.1109/ACCESS.2019.2963105>.
- Wang, Z., Ng, S.X., El-Hajjar, M., 2025. A 3D spatial information compression based deep reinforcement learning technique for UAV path planning in cluttered environments. IEEE Open J. Veh. Technol. 6, <http://dx.doi.org/10.1109/OJVT.2025.3540174>.
- Wang, L., Wang, K., Pan, C., Xu, W., Aslam, N., Nallanathan, A., 2022. Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing. IEEE Trans. Mob. Comput. 21 (10), 3536–3550. <http://dx.doi.org/10.1109/TMC.2021.3059691>.
- Wu, W., Zhang, L., Le, J., Lu, Z., 2025. Integrated method for multi-UAV task assignment and trajectory planning with deadlock based on Three-dimensional Dubins path. Sci. Rep. 15, 24152. <http://dx.doi.org/10.1038/s41598-025-09753-x>.
- Zhang, D., Xuan, Z., Zhang, Y., Yao, J., Li, X., Li, X., 2023. Path planning of unmanned aerial vehicle in complex environments based on state-detection twin delayed deep deterministic policy gradient. Machines 11 (1), 108. <http://dx.doi.org/10.3390/machines11010108>.
- Zhang, W., Zhang, S., Wu, F., Wang, Y., 2021. Path planning of UAV based on improved adaptive grey wolf optimization algorithm. IEEE Access 9, 89400–89411. <http://dx.doi.org/10.1109/ACCESS.2021.3090776>.
- Zhao, G., Wang, Y., Mu, T., Meng, Z., Wang, Z., 2024. Reinforcement learning assisted multi-UAV task allocation and path planning for IIoT. IEEE Internet Things J. 11 (16), 26766–26777. <http://dx.doi.org/10.1109/JIOT.2024.3370152>.
- Zheng, D., Zhang, Y.-f., Li, F., Cheng, P., 2023. UAVs cooperative task assignment and trajectory optimization with safety and time constraints. Def. Technol. 20, 149–161. <http://dx.doi.org/10.1016/j.dt.2022.01.011>.
- Zhou, H., Long, Y., Gong, S., Zhu, K., Hoang, D.T., Niyato, D., 2023. Hierarchical multi-agent deep reinforcement learning for energy-efficient hybrid computation offloading. IEEE Trans. Veh. Technol. 72 (1), 986–1001. <http://dx.doi.org/10.1109/TVT.2022.3202525>.